

Solver taxonomy: analytical: exact answer, including brute force (not just formulae); numerical/approximate: meta-heuristics; observational: data-driven/ML.

Random Search: each iteration = completely independent solution from scratch, no memory, no incremental moves. Return best seen.

Hill Climbing: Simple/First-Ascent: accept first improving neighbour (fast); Steepest-Ascent: evaluate all neighbours, pick best (slower). A neighbour is a solution reachable by one small perturbation.

Simulated Annealing: local search like HC; accepts worse solutions with prob $e^{\Delta E/T}$. Cooling: logarithmic (slowest); linear (fixed ΔT per step); exponential (fixed fraction per step, fastest initial drop since a fraction of a high T is a large absolute decrease).

EA components: representation, fitness, initialisation, parent selection, variation operators, survivor selection.

Parent selection: Roulette Wheel: selection probability $\propto$ fitness; sensitive to fitness scaling, cannot handle negative fitness directly. Tournament: draw k individuals, select fittest; larger k = higher selective pressure, smaller k = more diversity. Preferred in practice: robust to fitness scale.

Survivor selection: Full generational replacement: all parents replaced by offspring; no elitism, high diversity. Steady-state: replace one (or few) individuals per step. Elitism: copy top n individuals unchanged to next generation; prevents losing best found. Age-based: individuals removed after fixed lifespan regardless of fitness; maintains diversity, prevents stagnation.

Invalid solutions: proportional penalty preserves gradient near constraint boundary; hard-zero loses all information about how far outside the feasible region a solution is.

Crossover vs mutation: standard GA: crossover = exploitation (recombines existing material), mutation = exploration (injects new material). GP inverts this: crossover = exploration (swaps entire subtrees, massive structural jump), mutation = exploitation (replaces one node with same-arity node, tiny tweak), since GP's symbol set is small and already covered in the initial population.

Binary: crossover: 1-point, 2-point, uniform; mutation: bit-flip.

Real-valued: crossover: arithmetical ($x' = \alpha x_1 + (1-\alpha)x_2$, weighted avg); heuristic ($x' = \alpha(x_2 - x_1) + x_2$, extrapolate beyond fitter parent x_2); BLX- α (sample uniformly from $[\min - \alpha d, \max + \alpha d]$ where $d = |x_2 - x_1|$, α controls range extension). Mutation: uniform (draw new value from full valid range, no relation to current); Gaussian (add $\mathcal{N}(0, \sigma)$ perturbation, finer step-size control since most changes are small); creep (Gaussian with small fixed σ); random reset (draw from valid range, like uniform).

Permutation: crossover: OX (copy segment, fill remaining positions in order from other parent, preserving relative order); PMX (copy segment, use positional mapping to resolve conflicts); Cycle XO (identify cycles of positions that must move together to preserve absolute positions). Mutation: swap, insert, inversion, scramble (randomly reorder a subsequence).

GP representation: genotype = parse tree; terminals = leaves, functions = internal nodes. Closure: every function must accept any value a terminal can produce. Protected division: return 1 (or 0) when divisor = 0, so division is always defined. Strongly typed GP: explicit type annotations solve closure more cleanly.

GP initialisation: Full: all branches grow to max depth d (bushy, uniform). Grow: at each node choose from functions or terminals (variable depth, irregular shape). Ramped Half-and-Half: mix of Full and Grow across a range of depths; preferred since it produces

diverse sizes and shapes.

Bloat: excessive tree growth without fitness improvement. Causes: (1) introns: neutral code pads trees, protecting them against destructive operators; (2) search-space bias: more large programs than small at any fitness level; (3) crossover bias: crossover produces many small unfit offspring eliminated by selection, raising average size of survivors. Countermeasures: size penalty in fitness; depth/size limits; double tournament (run d_f fitness tournaments to get d_f winners, then one size tournament over those winners; size pressure always applied, not just as tiebreaker); context-preserving crossover (choose cut point so exchanged subtrees are of similar size).

GP overfitting: GP searches a huge unconstrained space and tends to overfit. Mitigations: Occam's razor (prefer simpler programs of equal fitness); validation set (evaluate on held-out data to detect overfitting).

Linear GP: genotype = sequence of register-based instructions (e.g. $r_1 = r_2 + r_3$); execution is sequential, unlike tree GP's recursive evaluation. Mutation: replace an instruction or operand. Crossover: two-point swap of code segments. Inactive instructions (not used to compute output) act as introns.

Cartesian GP: genotype = flat integer list encoding a directed acyclic graph; each node has function genes (which operation) and connection genes (which prior nodes to use as inputs). Levels-back parameter l limits how far back connections can reach, preventing cycles. Many nodes may be inactive (not connected to output) without changing the genotype.

Grammatical Evolution (GE): grammar + flat codon vector; each codon selects a production rule via codon mod #rules; wraps around if codons run out. Problem: same list position means a different decision across individuals, making crossover poorly aligned. DSGE fixes this by giving each grammar non-terminal its own dedicated gene, so position within a gene always governs the same type of decision.

ES: self-adaptive σ , step sizes co-evolved alongside object variables. (1+1)-ES: one parent, one offspring, greedy selection; 1/5 rule adapts σ . (μ, λ)-ES: $\lambda > \mu$, survivors from offspring only, allows forgetting bad solutions. ($\mu + \lambda$)-ES: survivors from parents and offspring, elitist, cannot escape bad solutions as easily.

CMA-ES: problem with simple ES: a single σ assumes a spherical mutation distribution; inefficient on elongated/correlated landscapes. CMA-ES adapts the full covariance matrix C of the mutation distribution $\mathcal{N}(\mu, \sigma^2 C)$, so the mutation ellipsoid rotates and scales to match the landscape shape. Generational loop: (1) sample λ offspring from $\mathcal{N}(\mu^{(g)}, \sigma^2 C^{(g)})$; (2) evaluate, select best μ_{best} ; (3) update mean $\mu^{(g+1)}$; (4) update $C^{(g+1)}$ using best individuals relative to *old* mean $\mu^{(g)}$ (not new mean, to measure actual steps taken rather than steps from the new centre); (5) adapt σ separately via cumulative step-size adaptation (consecutive steps in same direction $\rightarrow$ increase σ ; erratic/backtracking $\rightarrow$ decrease); (6) repeat. Note: C controls shape (ellipsoid orientation/correlations), σ controls overall scale: adapted separately since they capture different things.

NES: treats the population as a distribution $p(x|\theta)$ (parameterised by $\theta = \{\mu, \sigma, C, \dots\}$) and maximises expected fitness $\mathbb{E}[f(x)]$ via the natural gradient $\Delta\theta = F^{-1} \nabla_{\theta} \mathbb{E}[f(x)]$, where F is the Fisher Information Matrix. Standard gradient is biased by coordinate choice (same Euclidean step has different effects depending on where you are in parameter space); natural gradient corrects for this by moving in the direction of steepest ascent in distribution space (KL sense), making updates parameterisation-invariant. Unlike CMA-ES, NES uses *all* individuals weighted by fitness, including bad

ones: negative-weight bad solutions contribute information about where not to go, giving a more complete gradient estimate.

DE mutation: $v_i = x_1 + \gamma(x_2 - x_3)$. Difference vector self-adapts implicitly with fixed γ : shrinks as population converges (individuals cluster), grows when spread out.

DE crossover (binomial): $u_{ik} = v_{ik}$ if $r < r_{\text{cross}}$ or $i = i_{\text{rand}}$, else x_{ik} . The i_{rand} condition guarantees at least one gene always comes from the mutant.

DE selection: deterministic: keep trial u_i if $f(u_i) \geq f(x_i)$, else keep x_i .

DE notation: DE/x/y/z: base vector / #difference vectors / crossover type. Default: DE/rand/1/bin. Variants: DE/best/1/bin (higher selective pressure, lower diversity); more difference vectors (higher diversity); random or linear γ schedule.

PSO velocity: $v^{(j)} = v_{\text{in}} + v_{\text{cog}} + v_{\text{soc}}$, where $v_{\text{in}} = k_{\text{in}} \cdot v^{(j-1)}$ (momentum/inertia); $v_{\text{cog}} = k_{\text{cog}}(x_{\text{best}} - x^{(j-1)})$ (pull toward personal best); $v_{\text{soc}} = k_{\text{soc}}(x_{\text{gbest}} - x^{(j-1)})$ (pull toward swarm best).

PSO velocity control: clamping $[-v_{\text{max}}, v_{\text{max}}]$; decreasing inertia (explore $\rightarrow$ exploit over time); constriction coefficient $\chi = 2/|2 - c - \sqrt{c^2 - 4c}|$ with $c = k_{\text{cog}} + k_{\text{soc}} > 4$ (stabilises convergence).

PSO diversity: swarm activity $sa = \frac{1}{n} \sum |x_i(t) - x_i(t-1)|$. Mechanisms: random immigrants; forgetting (reset personal best); predator-prey (predator attracted to global best, prey repelled by distance-based term).

Binary PSO: same vel. update; $x_i = 1$ if $U(0, 1) < \sigma(v_i)$, else 0.

Permutation PSO: random keys encoding.

ACO order: (1) all ants build complete solutions; (2) evaporate all pheromone; (3) deposit proportional to solution quality. Deposit must come after building since deposit amount depends on fitness.

ACO move probability: $p_{ij}^k = \frac{\tau_{ij}^\alpha \cdot \eta_{ij}^\beta}{\sum_h \tau_{ih}^\alpha \cdot \eta_{ih}^\beta}$, where α weights pheromone, β weights heuristic (e.g. $\eta_{ij} = 1/d_{ij}$). Pheromone update: $\tau(t+1) = (1-\rho)\tau(t) + \Delta\tau$. Pheromones initialised to a small positive value so ants can move from the start; heuristic η guides initial decisions when all τ are equal.

ACO variants: Ant System: all ants deposit Q/L_k ; Elitist: all deposit plus best-ever deposits extra $\Delta\tau^*$; Rank-Based: top- w ants deposit weighted by rank.

Co-evolutionary fitness: $f: X_1 \times \dots \times X_n \rightarrow E$, fitness depends on interaction with other individuals, not on the individual alone.

Competitive co-EA: solution = single individual from one X_i . One-population pathologies: cycling (rediscovers same solutions, RPS-style); disengagement (fitness gap too large, no useful gradient); overspecialization/focusing (mastery against certain opponents, brittle elsewhere); forgetting (lose ability to beat past opponents once they leave the population). Red Queen Effect: related but distinct, perpetual arms race with no net relative gain, not necessarily cycling.

Interaction schemes: round-robin: every pair plays once, $n(n-1)/2$ interactions. Single-elimination: $n-1$ interactions (each match eliminates one). Hall of Fame: individuals also play against best solutions from past generations, directly addressing forgetting.

Subjective vs objective fitness: subjective evaluated against current population (feasible); objective against all possible opponents (effectively infinite opponent space, infeasible in practice).

Two-population (S + T): avoids the dual conflicting role in one-population co-EA (must simultaneously be a good solution and a good test). T evolves to expose S's weaknesses; S evolves to pass T. Structural asymmetry when representations differ (e.g. sorting networks vs test inputs); symmetric in game-playing settings.

Cooperative co-EA: solution = tuple across all X_i 's. Credit assignment: cannot isolate one component's contribution; evaluate by pairing with sampled collaborators, producing a noisy fitness signal. Benefits: modularity + exponentially smaller projected search spaces ($n^k \rightarrow k \times n$).

Pareto dominance: x dominates x' if $\leq$ in all objectives and $<$ in at least one. P^* = non-dominated feasible set; PF^* = its image in objective space. Best value in any single objective is always on PF^* (nothing can dominate it in that objective).

MOEA design issues: fitness assignment: convert multiple objectives into a selectable signal (e.g. count how many individuals dominate you, giving a rank). Diversity preservation: spread solutions across PF^* using a density metric in objective space. Elitism: prevent good non-dominated solutions from being lost across generations.

Pareto front shapes: location in objective space depends on optimisation direction. Both minimised: front in lower-left region. Both maximised: upper-right. One min, one max (or vice versa): lower-right or upper-left. Shape (convex vs concave) is problem-dependent.

LOTZ: f_1 = leading ones, f_2 = trailing zeros; gaining a leading 1 costs a trailing 0 (conflict). Pareto front = $\{1^k 0^{n-k}\}$ for $k = 0 \dots n$, evenly structured and analytically known; useful for testing MOEA diversity preservation.

SEMO: archive of non-dominated solutions; each step: mutate random member, add offspring if non-dominated (remove any it dominates). Problems: no crossover; binary fitness signal (dominated / not); no diversity mechanism; unbounded archive.

NSGA-II (fixes SEMO): (1) fast non-dominated sorting: graded front ranks instead of binary dominated/not (fixes slow convergence); (2) crowding distance: for each objective, boundary solutions (best and worst in that objective) get ∞ ; interior solutions get $\sum_{\text{obj}} (f_{\text{next}} - f_{\text{prev}}) / (f_{\text{max}} - f_{\text{min}})$; less crowded preferred as tiebreaker (fixes poor diversity); (3) combined $2\mu \rightarrow \mu$ pool: parents compete against offspring, good parents survive their own children (fixes loss of solutions). Pipeline: init $P(\mu)$; each gen: generate $Q(\mu)$; merge $R = P \cup Q$; non-dominated sort; fill new P greedily by front, last partial front by crowding distance descending.

EC in ML: feature selection (2^N space, binary vector genotype); model topology (non-differentiable structural choices); model weights (alternative to backprop, no gradient needed); hyperparameters (sit outside the training loop).

Direct weight encoding: genotype = weight vector, phenotype = network; fitness = $1/(1+\text{MSE})$ (maps minimisation to $(0, 1]$). Problems: redundancy (many encodings produce same function, flat landscape); crossover incoherence (different networks develop different internal representations, swapping sub-vectors destroys structure); computational cost. Feedforward: only upper-triangle of weight matrix needed; recurrent: full matrix.

NEAT: three pillars. (1) Novel encoding with innovation numbers: connection gene = (from-node, to-node, weight, enabled/disabled, innov#). Innovation number is a global counter assigning a unique ID to each (from, to) pair; any individual with connection $u \rightarrow v$ carries the same number, whether inherited or independently mutated. Crossover aligns by innovation number, not list position: matching genes (same #) inherit from either parent at random; disjoint and excess genes inherit from the fitter parent. Add-node mutation: disable $A \rightarrow B$, insert node C , add $A \rightarrow C$ and $C \rightarrow B$ with fresh numbers. Disabled genes stay in the genome. (2) Speciation: $\delta = c_1 E/N + c_2 D/N + c_3 \bar{W}$ (excess / genome size, disjoint / genome size, avg weight diff on matching genes); assign to same species if $\delta < \delta_t$. Individuals compete within species,

protecting new topologies while weights tune before facing the full population. Fitness sharing $f'(i) = f(i) / \sum_j s(\delta(i, j))$ penalises over-large species. (3) Minimal initialisation: no hidden nodes at start; complexity added only when evolution finds it useful. NEAT only adds nodes and connections, never removes.

CPPN: function $F(x_1, y_1, x_2, y_2) = w$, outputs weight from node coordinates, assuming geometric regularity. Compact: stores the function, not n^2 weights. Evolved by: NEAT (network graph), GP (syntax tree), GE, or DSGE (see GP section).

DENSER: two-level architecture search. GA level: sequence of layers; crossover swaps layer blocks; mutation: add, replicate, remove layer. DSGE level: parameters within each layer type; crossover swaps same-category layer parameters; mutation: grammatical (change a rule choice) or integer/float (perturb numeric param). Weights trained by backprop after architecture is fixed. Does not uniformly outperform human-designed architectures.